I know there is a goodle plarfrom that can make fuzzer compete each other on binutil apps, or so. So you can send your fuzzer to this platfrom and they will run and assess it against the rest of the fuzzers they have

Thought for 19s >

You're thinking of **FuzzBench** (made by Google). It's basically "fuzzer benchmarking as a service": you add your fuzzer to their platform, they run standardized experiments at scale, and you get a report comparing it to other fuzzers. google.github.io +2

What the platform is (and isn't)

FuzzBench (the "fuzzers compete" one)

- Purpose: **compare fuzzers against each other** on the same set of real-world benchmarks, using consistent methodology. GitHub +2
- How it works (official flow): you **integrate** your fuzzer into the FuzzBench repo → it gets merged → their service runs experiments → a **published report** compares performance against existing fuzzers (per-benchmark + aggregate). google.github.io
- Benchmarks: there's a documented list of included targets; many are derived from OSS-Fuzz fuzz targets. google.github.io +1

OSS-Fuzz / ClusterFuzz (related, but different)

- **OSS-Fuzz** is continuous fuzzing for open-source projects (find bugs in projects over time), not primarily a "fuzzer tournament." solar.cs.ucl.ac.uk +1

- FuzzBench benchmarks are often based on OSS-Fuzz targets, but the goals differ: **FuzzBench = evaluation**, **OSS-Fuzz = ongoing bug-finding for projects**. solar.cs.ucl.ac.uk

About “binutils apps”

- FuzzBench’s standard benchmark set is documented; **binutils isn’t necessarily a default benchmark** in the main set, but the ecosystem absolutely evaluates on binutils-family tools (e.g., papers/bench suites integrating binutils utilities like `objdump`). google.github.io +2
- Practically: if you care specifically about binutils, you can still use the **FuzzBench tooling/infrastructure** to integrate those targets (either privately in your fork/local runs, or as a contributed benchmark if accepted). GitHub +2

How you’d “send your fuzzer” to it (high-level steps)

1. **Make your fuzzer runnable in their harness model** (usually Docker-based, consistent entrypoints, etc.). google.github.io +1
2. **Add an integration** for your fuzzer to the FuzzBench repository (their docs cover “Adding a fuzzer”). google.github.io +1
3. **Open a Pull Request** to the FuzzBench repo.
4. Once merged, the service **runs experiments and publishes a comparison report** against other fuzzers. google.github.io

Pro tips so your results are taken seriously

- **Be explicit about your “fairness” assumptions**: CPU cores, memory, dictionary usage, seed corpora, and whether you rely on special instrumentation.
- **Expect variance**: fuzzing results can be noisy; FuzzBench is designed to make comparisons more reproducible, but you should still interpret small differences carefully. Google Research +1

- If your goal is “best on binutils,” consider:
  - Running **local FuzzBench experiments** with a binutils benchmark you add yourself.
  - Comparing on both **coverage growth** and **bug/CRASH discovery** depending on what you’re optimizing for.

If you tell me (1) what base fuzzer you built on (AFL++, libFuzzer, honggfuzz, custom) and (2) what’s novel about yours (mutator, scheduler, power schedule, grammar, concolic, etc.), I can suggest the cleanest way to package it for FuzzBench-style evaluation and what metrics to emphasize.

```
URLs (for your reference)
https://google.github.io/fuzzbench/
https://github.com/google/fuzzbench
https://google.github.io/fuzzbench/reference/benchmarks/
https://research.google/pubs/fuzzbench-an-open-fuzzer-benchmarking-platform-an
https://opensource.googleblog.com/2020/03/fuzzbench-fuzzer-benchmarking-as-ser
https://bughunters.google.com/about/rules/open-source/oss-fuzz-reward-program-
```

Sources

How to install fuzz bench and add my fuzzer to it?
Do I need to install it?

Thought for 6s >

Yes — **you usually do need to install / run FuzzBench locally** at least for **integration + testing**. But **you don’t need to own Google’s infrastructure** to get “official” comparisons: once your fuzzer is integrated and accepted

upstream, FuzzBench can run it as part of their service (that part is “hosted”). google.github.io +2

Below is the practical workflow that most people follow.

Decide which path you want

Path A — “I just want to compare my fuzzer”

You:

1. install prerequisites
2. clone FuzzBench
3. integrate your fuzzer
4. run **local experiments** to verify it works
5. open a PR to FuzzBench (so it can be run in the service) google.github.io +2

Path B — “I only want to run locally”

You can run local experiments without contributing upstream, but then the results are “your local results,” not the official hosted comparison reports. Local runs are still very useful for debugging and iteration. google.github.io +1

What you need installed (local machine)

FuzzBench’s local workflow is Docker-based.

Minimum practical setup:

- **Docker** (required for builds + running experiments) google.github.io +1
- **Python 3** (FuzzBench scripts are Python; exact version can vary by doc/repo state) google.github.io +1
- **git** (to clone + PR)

If you plan to run cloud experiments yourself, that's a separate setup (Google Cloud project + APIs). google.github.io +1

Install / set up FuzzBench locally (basic)

1. Clone:

```
git clone https://github.com/google/fuzzbench.git
cd fuzzbench
```

GitHub

2. Follow "Getting started" prerequisites from the docs (Docker + Python, etc.). google.github.io
3. Sanity check (this is important because it catches naming/lint issues early):

```
make presubmit
```

The docs specifically mention using `make presubmit` to validate things like your fuzzer module name. google.github.io

---

Add your fuzzer to FuzzBench (what "integration" means)

FuzzBench expects each fuzzer to be a **module under** `fuzzers/` with a small amount of glue code and usually a Dockerfile/build steps.

1) Create a fuzzer directory

```
export FUZZER_NAME=<your_fuzzer_name>
mkdir -p fuzzers/$FUZZER_NAME
```

Your fuzzer name must be a valid Python module name (alphanumeric + underscores). google.github.io

2) Implement the expected fuzzer interface

The “Adding a new fuzzer” guide walks through the files you need in that directory (the fuzzer “runner”/integration code). google.github.io +1

Fastest way to do it correctly:

- copy an existing fuzzer integration that’s closest to yours (e.g., AFL++ integration) and adapt it.
  Example reference integration file: AFL++’s `fuzzer.py` in the repo.

  GitHub +1

3) Make sure your fuzzer can run in the FuzzBench model

FuzzBench runs experiments in containers and expects the fuzzer to:

- take a corpus/input directory
- run against a benchmark harness/target in the expected way
- output artifacts the framework expects (coverage/crashes/etc., depending on config)

The docs show the “contract”; you’re basically implementing that contract.

google.github.io +1

Test it locally (before you PR)

FuzzBench has a documented “run local experiment” flow using `run_experiment.py`, which spins up a dispatcher container and runs the experiment end-to-end. google.github.io

Typical workflow:

1. Pick a small experiment config (short duration, few trials)
2. Run:

```
python3 experiment/run_experiment.py <path_to_config.yaml>
```

(Exact path/args are in the local experiment doc; follow that page for the canonical command and config format.) google.github.io

3. View the generated report (the doc covers where outputs/reports go).

google.github.io

Get it benchmarked “against the rest” (hosted service)

To have it run and appear alongside other fuzzers in the FuzzBench service, you generally:

1. ensure local experiment works
2. run presubmit
3. open a PR to `google/fuzzbench`

FuzzBench is explicitly designed for community contributions of fuzzers.

GitHub +1

Common gotchas (so you don’t waste days)

- **Reproducibility:** if your fuzzer depends on a custom kernel, special privileges, or external services, it's harder to integrate (containers are constrained).
- **Licensing:** your fuzzer code and dependencies need to be compatible with the repo's contribution expectations (practically: avoid "download random binary blob" patterns).
- **Fair comparisons:** if your fuzzer needs custom flags/dictionaries/seeds, document and integrate them in a way that matches the evaluation methodology.

Direct answer to your question: "Do I need to install it?"

- **To submit / add your fuzzer:** yes, you should set it up locally to integrate and test. google.github.io +1
- **To get it compared by the hosted service:** you don't need to run the service infrastructure yourself, but you *do* need to contribute an integration that works. GitHub +1

If you tell me 3 quick things, I can map your exact integration steps

1. Is your fuzzer **AFL++-based**, **libFuzzer-based**, **honggfuzz-based**, or totally custom?
2. Does it require special instrumentation (LLVM pass, QEMU, Frida, DynamoRIO, etc.)?
3. What OS are you on (Ubuntu version / macOS / Windows+WSL)?

I'll point you to the closest existing fuzzer template in the repo and the exact files you'll likely need to touch.

URLs (listed, not inline)

- https://google.github.io/fuzzbench/ ↗
- https://google.github.io/fuzzbench/getting-started/ ↗
- https://google.github.io/fuzzbench/getting-started/adding-a-new-fuzzer/ ↗
- https://google.github.io/fuzzbench/running-a-local-experiment ↗
- https://google.github.io/fuzzbench/running-a-cloud-experiment/setting-up-a-google-cloud-project/ ↗
- https://github.com/google/fuzzbench ↗
- https://github.com/google/fuzzbench/blob/master/fuzzers/aflplusplus/fuzzer.py ↗